

TickExp Project Specification

The Golden Build - v1.1

TickExp is a high-fidelity, interactive booking ecosystem built with Jetpack Compose and a modern Clean Architecture pattern. This document outlines the core systems that make the "Golden Build" functional.

1. The Dynamic Backend Engines (Movies, Trains, Buses)

TickExp does not just use static text; it is powered by three distinct backend data sources routed through our secure Vercel Proxy (`https://tickexp-backend.vercel.app/api/`):

- **TMDB Movie Engine**: Connects to TheMovieDB global database to fetch real-time "Now Playing" posters, ratings, and plot summaries.
- **RapidAPI IRCTC Integration**: The train booking module routes through a RapidAPI endpoint designed to fetch real-time Indian Railways schedules and PNR statuses.
- **RedBus B2B Simulator**: The bus module implements a custom RedBus aggregator fake API. It is engineered to simulate high-speed dynamic bus inventory allocation for academic and demo purposes, avoiding third-party rate limits while functioning exactly like a real-world B2B integration.
- **Async Processing**: Uses **Retrofit 2** and **Kotlin Coroutines** on the Android client to concurrently fetch from all three engines without lagging the UI.

2. Geo-Velocity Location System (The GPS)

The app uses real hardware sensors to personalize the experience:

- **FusedLocationProvider**: Utilizes Google Play Services to triangulate the user's exact coordinates.
- **Geocoding**: Converts raw Latitude/Longitude into a human-readable city name (e.g., "Chennai", "Mumbai").
- **Dynamic Filtering**: Once a city is detected, the `TheatreViewModel`` automatically filters and loads nearby cinema halls and showtimes specific to that region.

3. Financial Infrastructure (The Wallet)

A real-world economy simulation:

- **Razorpay SDK**: Integrated the official Razorpay Android SDK. While currently in **Test Mode** for the demo, the cryptographic signature and payment intent flows are identical to a production app.
- **Persistent Wallet**: Uses a local persistence layer to manage the user's balance. Recharges via

TickExp Project Specification

The Golden Build - v1.1

Razorpay update the balance, and bookings deduct from it.

- **Velocity Points**: A loyalty logic that rewards users with 10% of their ticket price in points for every successful transaction.

4. Ticket Generation & Security

Every booking produces a tangible digital asset:

- **QR Logic**: Uses the `Zxing` algorithm to generate unique QR codes containing a hashed string of the Booking ID + Seat Map.
- **PDF Export Engine**: A custom `TicketExporter` utility that renders a high-resolution ticket layout and exports it as a shareable PDF file.
- **Persistence**: User identity (Name, Phone, Image) is stored in `SharedPreferences`, ensuring the ticket owner's details are always accurate.

5. UI Architecture (The Design)

- **Glassmorphism**: Built using custom `graphicsLayer` modifications and `Brush` gradients to achieve the "Frosted Glass" look.
- **Hardware Acceleration**: Uses `drawBehind` and `blur` modifiers optimized for modern Android GPUs.
- **Navigation**: A multi-stack `NavHost` that handles 16+ distinct screen states with smooth `AnimatedVisibility` transitions.

TickExp is a production-ready blueprint of what a modern, fast, and secure booking application looks like.